



Конспект: Оптимизация многопоточных программ на Go

Время изучения: 15–20 минут

Автор: Матвей Ефимов, эксперт в Go, Python, SQL

Содержание

1. [Введение](#)
2. [Лучшие практики оптимизации](#)
 - [Ограничение числа горутин](#)
 - [Выбор типа синхронизации](#)
 - [Неблокирующие подходы](#)
 - [Управление жизненным циклом](#)
 - [Сокращение накладных расходов](#)
3. [Сводка-чек-лист](#)

1. Введение

Этот конспект предназначен для оперативного повторения ключевых приемов оптимизации конкурентного кода на Go. Он поможет системно выявлять и устранять типовые сложности — избыточную синхронизацию, contention, избыточное число горутин — а также быстро выбирать нужные инструменты под конкретную задачу. Материал организован по направлениям оптимизации, содержит минимальные, но показательные примеры кода и резюмирован в удобный чек-лист.

2. Лучшие практики оптимизации

Ограничение числа горутин



Проблема: Чрезмерное число горутин приводит к высокому потреблению памяти, загрузке планировщика и деградации производительности.

Решения: Семафор на буферизованных каналах

```
sem := make(chan struct{}, 20) // максимум 20 горутин
for _, task := range tasks {
    sem <- struct{}{}
    go func(t Task) {
        defer func() { <-sem }()
        process(t)
    }(task)
}
```

Пример: Worker pool

```
jobs := make(chan Task)
for i := 0; i < 5; i++ {
    go func() {
        for job := range jobs {
            process(job)
        }
    }()
}
for _, task := range tasks {
    jobs <- task
}
close(jobs)
```



Рекомендации:

- Всегда оценивайте необходимость каждой новой горутины.



- Используйте фиксированное количество воркеров для управляемого параллелизма.
- Избегайте запуска горутин в цикле без ограничения их числа.



Выбор типа синхронизации



Проблема: Избыточная или неадекватная синхронизация ведёт к блокировкам, contention, деградации производительности.

Механизм	Когда использовать	Пример
Канал (chan)	Обмен данными между горутинами	ch <- value
Mutex	Защита разделяемого ресурса	mu.Lock() ... mu.Unlock()
RWMutex	Разделение чтения/записи	mu.RLock() ... mu.RUnlock()
Atomic	Для счётчиков и флагов	atomic.AddInt32(&x, 1)
Confinement	Изоляция доступа извне (actor model)	Вся работа внутри горутины
WaitGroup	Ожидание завершения набора горутин	wg.Add() / wg.Wait()
Context	Завершение/отмена по тайм-ауту, сигналу	ctx, cancel := context.WithTimeout(...)

Примеры:

```
// Mutex
var mu sync.Mutex
mu.Lock()
sharedVal++
mu.Unlock()

// RWMutex
var mu sync.RWMutex
func Read(k string) string {
```



```
mu.RLock()
defer mu.RUnlock()
return data[k]
}
```



Рекомендации:

- Для обмена сообщениями между горутинами используйте каналы.
- Для защиты данных — Mutex/RWMutex.
- Для простых счётчиков — атомарные операции (sync/atomic).
- Для локализации данных — паттерн confinement.
- Для ожидания завершения — WaitGroup.
- Для отмены/тайм-аутов — Context.

Неблокирующие подходы



Цель: Сократить время ожидания, повысить отзывчивость приложения, избежать deadlock.

Select с default:

```
select {  
  case ch <- msg:  
    // успешно отправлено  
  default:  
    // канал занят — пропускаем  
}
```

Select с тайм-аутом:

```
select {  
  case result := <-ch:  
    use(result)  
  case <-time.After(50 * time.Millisecond):  
    log.Println("Timeout")  
}
```

Проверка состояния перед записью:

```
if len(queue) < cap(queue) {  
  queue <- item  
}
```



Рекомендации:

- Используйте select с default для избежания блокировок.
- Добавляйте тайм-ауты там, где критична скорость реакции.
- Не допускайте бесконечного ожидания ресурсов.

Управление жизненным циклом горути



Проблема: Утечки горути = накопление «мусора» в памяти, снижение производительности и отказоустойчивости.



Решения:

- Пользуйтесь `defer` для автоматического освобождения ресурсов
- Для ожидания завершения горути используйте `sync.WaitGroup`
- Управляйте временем жизни через `context.Context`
- Закрывайте каналы только владельцем

Примеры:

```
ctx, cancel := context.WithCancel(context.Background())
go func() {
    for {
        select {
        case <-ctx.Done():
            return
        case msg := <-in:
            handle(msg)
        }
    }
}()
// При необходимости остановки:
cancel()
```

```
var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    doWork()
}()
wg.Wait()
```



Рекомендации:

- Контролируйте выход из горутины через `ctx`, флаг или закрытие канала.
- Не держите горутины «в подвешенном состоянии»: каждая должна иметь условие для завершения.
- Не забывайте `defer wg.Done()` в начале горутины.

Сокращение накладных расходов



Проблема: Частое создание объектов (например, буферов) увеличивает нагрузку на GC и замедляет приложение.



Решения:

- Используйте `sync.Pool` для пуллинга временных структур и буферов.

```
var bufPool = sync.Pool{
    New: func() interface{} {
        return make([]byte, 1024)
    },
}

func handler() {
    buf := bufPool.Get().([]byte)
    defer bufPool.Put(buf)
    // работа с буфером
}
```



Не используйте `sync.Pool` для постоянного хранения — только для часто переиспользуемых объектов.



Рекомендации:

- Применяйте `sync.Pool` для объектов, которые часто создаются и быстро возвращаются в пул.
- Всегда измеряйте эффект (до/после профилирования).
- Не переусложняйте архитектуру преждевременной оптимизацией.



Сводка-чек-лист

№	Рекомендация
1	Не запускайте слишком много горутин без ограничения — используйте семафоры или worker pool
2	Подбирайте механизм синхронизации исходя из задачи: данные — Mutex, обмен — каналы
3	Используйте атомарные операции для простых флагов и счётчиков
4	Применяйте паттерн confinement для изоляции доступа к данным
5	Избегайте блокировок: применяйте select с default или тайм-аутом
6	Контролируйте жизненный цикл горутин через Context и WaitGroup
7	Не оставляйте горутин в подвешенном состоянии — каждой нужен путь к завершению
8	Всегда используйте defer для освобождения ресурсов (в том числе wg.Done)
9	Повторно используйте объекты через sync.Pool, чтобы уменьшить аллокации
10	Профилируйте производительность перед оптимизацией — не оптимизируйте вслепую



Совет: Держите эту шпаргалку под рукой во время практики — она помогает быстро вспомнить проверенные решения и избежать частых ошибок конкуренции в Go.